

Por qué existen las herramientas: contexto más allá del prompt

Claude Certified Developer, Foundation (CCDV-F) · Masterclass UST

ES Sesión 10 de 12 · Notas en español (es-ES)

La sesión 10 (24 de agosto de 2026) arrancó con un recuento de la **gestión de contexto**. El planteamiento de Vijay: toda aplicación GenAI o agentic tiene que preparar un canal de contexto, enviarlo al LLM y consumir la respuesta. La ingeniería de prompts es una vía. El **uso de herramientas** es otra: el modelo puede pedir datos en vivo o propietarios que nunca estuvieron en el entrenamiento.

Para qué sirve este capítulo

Antes del esquema de la API Messages, hace falta el motivo de producto. Claude no conoce por magia el tiempo en Bangalore, un precio de acción en vivo o un registro de empleado en un sistema UST. Esos hechos viven fuera del modelo. Una herramienta es una función que posee vuestra aplicación; Claude decide *cuándo* llamarla y con qué argumentos; vuestro código la ejecuta.

INTERFAZ DE AULA FRENTE A LA DOCUMENTACIÓN

Los cuadernos, nombres de clases del SDK e IDs de modelo de agosto de 2026 pueden haber cambiado. Tomad **tool use** en docs.claude.com como fuente de verdad para el examen.

Cinco formas de enriquecer el contexto

El formador enumeró técnicas que sirven al mismo trabajo: que la petición que ve el modelo sea más rica y precisa que una frase cruda del usuario.

Técnica	Qué aporta al contexto	Ángulo típico de examen
Ingeniería de prompts	Instrucciones estructuradas, ejemplos, roles	Cómo escribís el mensaje, no cómo traéis datos
Uso de herramientas	Resultados de funciones en tiempo real o propietarias	El modelo pide una herramienta; la app la ejecuta
RAG	Fragmentos recuperados de un corpus	No volquéis un PDF de 10.000 páginas en un prompt
MCP	Acceso normalizado a herramientas y datos de terceros	Integráis una vez; reutilizáis en varios clientes
Memoria	Turnos previos o estado más duradero	Un chat conversacional necesita diseño explícito de memoria

Son complementarias, no rivales. Un agente de contratación puede usar prompts para el tono, herramientas para APIs de RR. HH., RAG para PDFs de política, MCP para un ATS de proveedor y memoria para que un objetivo de reclutamiento de una semana no se reinicie en cada turno.

Qué es una herramienta (y qué no es)

- Una herramienta es una **definición de función** más un **esquema** que enseña a Claude el nombre, cuándo llamarla (descripción) y qué argumentos JSON pasar.

- La función corre en *vuestro* proceso: donde esté el cliente Anthropic o la sesión de Claude Code. El LLM no ejecuta Python en el lado de Anthropic.
- Cargas típicas: fecha/hora actual, meteorología, cotizaciones, búsquedas de empleados, lecturas de base de datos — lo que los pesos no pueden saber.
- No es lo mismo que meter la respuesta en el prompt de sistema. Los prompts son estáticos en ese turno; las herramientas se invocan cuando el modelo las juzga pertinentes.

Vocabulario del formador: «Claude Code» frente a Claude

En esta sesión Vijay usó **Claude Code** para la *aplicación de IA* (el cliente que guarda funciones, construye mensajes y habla con la API) y **Claude** para el LLM. Es taquigrafía de aula. En la documentación de producto, Claude Code es el agente de terminal/IDE; el cliente de Messages es el SDK de Anthropic. Para el examen, tened claro el bucle: *el cliente ejecuta; el modelo solo decide*.

Dos SDK, un mismo bucle

El grupo había usado `pip install anthropic` (API Messages). El formador señaló un segundo stack: el **Claude Agent SDK** (antes citado como Claude Code SDK). El uso manual de herramientas deja visible cada apretón de manos. El Agent SDK sigue haciendo ese apretón; oculta el código repetitivo para que configuréis herramientas y un objetivo en lugar de escribir el bucle.

1. El prompt del usuario más la lista de esquemas va a Claude.
2. Claude responde en texto o devuelve una petición **tool_use** (motivo de parada `tool_use`).
3. La aplicación ejecuta la función coincidente con los argumentos que dio el modelo.
4. El resultado vuelve como mensaje de **usuario** con un bloque **tool_result**.
5. Claude produce la respuesta final (o pide otra herramienta).

Por qué no adjuntar todas las herramientas en cada llamada

Podéis registrar muchas; los esquemas son relativamente pequeños. El riesgo no es tanto el volumen de tokens como la **colisión**. Si dos descripciones se solapan, Claude puede elegir la función equivocada. Si ya sabéis que este turno solo necesita una herramienta de navegador, adjuntad esa. Nombres distintos y descripciones específicas son la superficie de control que el modelo lee de verdad.

DATOS DE CLIENTE

No conectéis sistemas de producción de clientes UST a los cuadernos de aula. Funciones de demostración (reloj, APIs públicas) bastan para aprender el apretón de manos.

Cómo encaja con los capítulos siguientes

El capítulo 02 recorre el apretón de manos de la API Messages. El 03 trata MCP como la versión reutilizable y de terceros de la misma idea. El módulo 11 envuelve la recuperación como otra técnica de contexto. El

módulo 12 muestra herramientas *integradas* del Agent SDK (Bash, Read, Glob) que habilitáis en lugar de escribirlas.

Documentación oficial

- **Visión general de tool use**

<https://docs.claude.com/en/docs/agents-and-tools/tool-use/overview>

Bucle canónico: esquemas, tool_use, tool_result.

- **API Messages**

<https://docs.claude.com/en/api/messages>

Dónde se adjuntan las herramientas a una petición.

- **Documentación de Claude Code**

<https://code.claude.com/docs/en/overview>

Superficie de producto que el formador comparó con «la app de IA».

- **Inicio de Claude Docs**

<https://docs.claude.com/en/home>

Índice si una ruta de cuaderno grabada está obsoleta.

Study notes distilled from the UST CCD Foundation masterclass. Prefer Anthropic's current docs over session anecdotes. Not official exam material.